

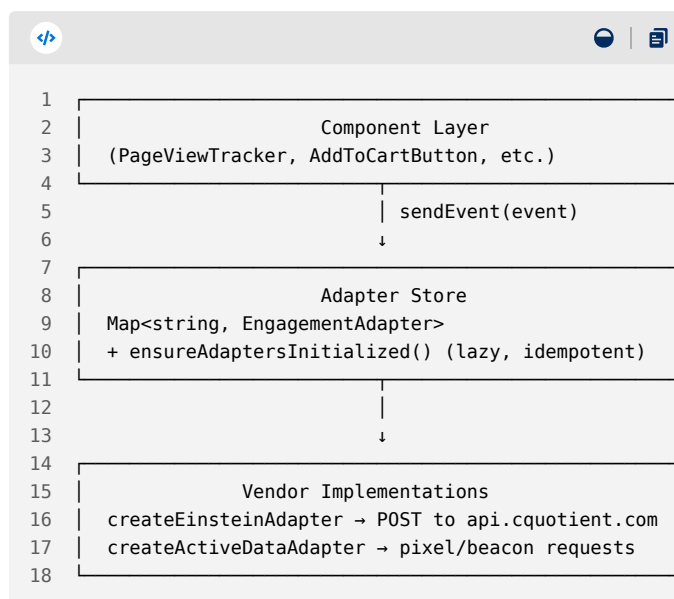
Engagement Adapter Pattern

The adapter pattern decouples analytics components from vendor-specific event APIs. Components call a generic interface and the adapter translates those calls into the vendor's API. Storefront Next provides two built-in adapters for analytics event tracking that use Einstein and Active Data.

Adapter	Purpose	Config key
Einstein	Analytics event tracking (viewProduct, addToCart, etc.)	engagement.adapters.einstein
Active Data	Analytics event tracking (dwac beacon)	engagement.adapters.activeData

Both implement the `EngagementAdapter` interface and are registered in a shared store.

Architecture



Contents

Architecture

[File Structure](#)[Adapter Interface](#)[Adapter Store](#)[Lazy Initialization](#)[Configuration](#)[Testing](#)[Add an Engagement Adapter](#)

[Try for free](#)

```
1 src/lib/adapters/  
2   └─ index.ts                # Re-exports store.  
3   └─ engagement/  
4       └─ types.ts           # EngagementAdapter  
5       └─ store.ts           # addAdapter, getAc  
6       └─ einstein.ts        # createEinsteinAd  
7       └─ active-data.ts     # createActiveData/  
8       └─ register.ts        # initializeEngager  
9       └─ initialize.ts       # ensureAdaptersIn:  
10      └─ einstein-config.ts  # validateEinstein(  
11      └─ utils.ts            # hasConsent helper
```

Adapter Interface

```
1 // src/lib/adapters/engagement/types.ts  
2 export interface EngagementAdapter extends EventAdapter {  
3   name: string;  
4   sendEvent?: (  
5     event: AnalyticsEvent,  
6     siteInfo?: EventSiteInfo,  
7     consentPreferences?: ConsentPreferences,  
8   ) => Promise<unknown>;  
9   send?: (url: string, options?: RequestInit) => Promise<unknown>;  
10 }
```

Adapter Store

A `Map<string, EngagementAdapter>` with functional accessors:

```
1 addAdapter("einstein", adapter); // Register  
2 getAdapter("einstein"); // Retrieve by name  
3 getAllAdapters(); // All registered adapters  
4 removeAdapter("einstein"); // Unregister
```

Lazy Initialization

Adapter code is dynamically imported to stay out of the initial bundle:

```
1 // src/lib/adapters/engagement/initialize.ts  
2 export async function ensureAdaptersInitialized(appConfig) {  
3   if (getAllAdapters().length > 0) return; // Already initialized  
4   const { initializeEngagementAdapters } = await import('einstein-adapter');  
5   initializeEngagementAdapters(appConfig);  
6 }
```

The dynamic `import()` means that the Einstein or Active Data implementation modules are code-split into a separate chunk.



```
1 engagement: {
2   adapters: {
3     einstein: {
4       enabled: true,
5       host: 'https://api.cquotient.com',
6       einsteinId: '<your-einstein-id>',
7       siteId: '<your-site-id>',
8       realm: '<realm>',
9       isProduction: false,
10      consentCategory: 'C0004',
11      eventToggles: { view_product: true, cart_i
12    },
13    activeData: {
14      enabled: true,
15      host: '<host>',
16      siteUUID: '<uuid>',
17      consentCategory: 'C0002',
18      eventToggles: { view_product: true, cart_i
19    },
20  },
21 },
```

If an adapter's `enabled` flag is `false`, it isn't registered.

Testing

Mocking Adapters in Tests

```
1 import { addAdapter, removeAdapter } from "@lib/adapters"
2
3 const mockAdapter: EngagementAdapter = {
4   name: "mock-einstein",
5   sendEvent: vi.fn().mockResolvedValue(undefined),
6 };
7
8 beforeEach(() => addAdapter("einstein", mockAdapter));
9 afterEach(() => removeAdapter("einstein"));
```

Testing Initialization

```
1 import { resetAdaptersInitialization } from "@lib/adapters"
2
3 afterEach(() => resetAdaptersInitialization()); // Clear
```

Add an Engagement Adapter

1. Create `src/lib/adapters/engagement/your-adapter.ts` with a factory function returning `EngagementAdapter`.

[Try for free](#)

```
engagement.adapters.yourAdapter in  
config.server.ts.
```

DID THIS ARTICLE SOLVE YOUR ISSUE?

Let us know so we can improve!

[Share your feedback](#)

DEVELOPER CENTERS

Heroku
MuleSoft
Tableau
Commerce Cloud
Lightning Design System
Einstein
Quip

POPULAR RESOURCES

Documentation
Component Library
APIs
Trailhead
Sample Apps
AppExchange

COMMUNITY

Trailblazer Community
Events and Calendar
Partner Community
Blog
Salesforce Admins
Salesforce Architects

© Copyright 2026 Salesforce, Inc. [All rights reserved](#). Various trademarks held by their respective owners. Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#) [Terms of Service](#) [Privacy Information](#) [Responsible Disclosure](#) [Trust](#) [Contact](#) [Use of Cookies](#)
[Cookie Preferences](#)  [Your Privacy Choices](#)

